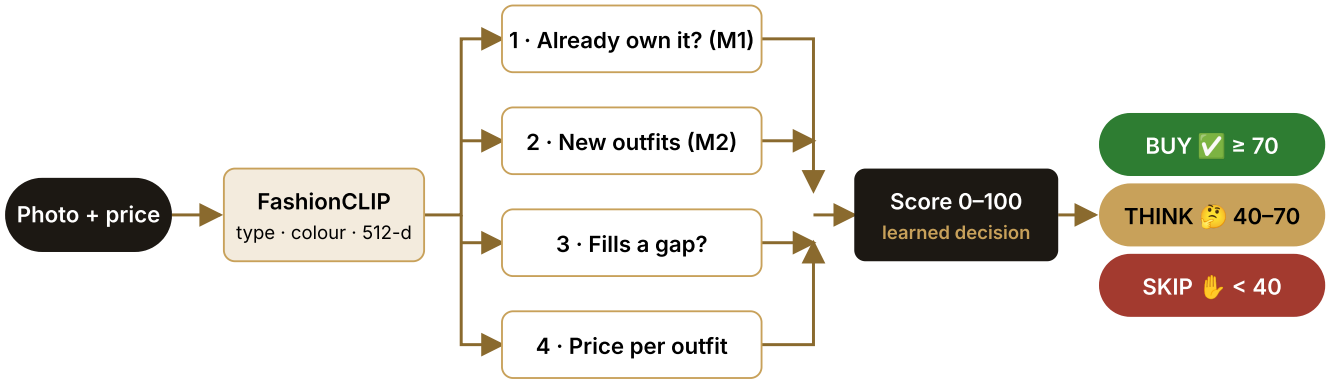
BO5 · Smart Buy

"Should I buy this piece?" — owner Aziz · M6 Smart Buy / what-if · skills M13 Partner links, M18 Size & fit

> **In one sentence:** the user photographs a piece in a shop (or describes it) with its price; Smart Buy checks it against **their own wardrobe** and answers **BUY ✅ / THINK 🤔 / SKIP ✋** with a score, the reasons, the right size and where to buy it locally.

1. The pipeline



then: explanation · size (M18) · local shops + promo (M7 / M13) · what-if

2. How the score is made

Criterion	Weight	From	Meaning
New outfits	40 %	M2	Outfits it unlocks with your clothes (8+ = full marks).
Gap fill	25 %	M1 gaps	It is one of the essentials you're missing.
Novelty	20 %	M1	1 − similarity to what you own (another colour counts half).
Price value	15 %	price	Cost per new outfit vs 15 TND target.

- **Verdict:** score ≥ 70 → BUY · 40–70 → THINK · < 40 → SKIP. Similarity ≥ 0.85 → duplicate → SKIP.
- **Too many of the same type** (≥ 3): BUY is capped at THINK.
- **Decision mode:** `learned` (logistic regression, 86 % accuracy) or `rules`. Personal goals move the score up to ±15.
- **Close call** (±5 points, or photo confidence < 0.5) → one question: "Do you need it for an event?". The answer goes back with its `question_key`.
- **XAI:** each factor shows how much it pushed toward BUY (+) or SKIP (−); **what-if** says what would change the answer.

3. After the verdict

- **Explanation** in plain words (explainer.py).
- **Size** (M18): from body numbers or "I wear M at Zara, tight on the shoulders" + the brand's size chart.
- **Where to buy** (M7 local catalogue + M13): Tunisian shops, TND prices, promo codes, cheaper alternatives.
- **Wishlist**: save a THINK piece, get told when the price drops.
- **30-day check-in**: "did you wear it? any regret?" → feeds learning and the bandit.
- **Shopping list**: the wardrobe gaps, ready to buy.

4. How it connects

With	How
Orchestrator (BO8)	Tool `smartbuy(probe_image, price_tnd, answer?)`. "shoul i by a blue costume with 320dt" → `custom:blazer:blue`, 320. The reply is written from its facts: "BUY ✅ — blue blazer, score 72/100..."
M1 wardrobe (BO3/BO4)	Your real pieces + similarity (real, mock as fallback).
M2 recommender (BO1/BO2)	New outfits with the piece locked.
Wardrobe gaps	`shared/wardrobe_gaps.py` on the M1 wardrobe (replaces the M5 mock).
M7 local catalogue (BO6)	Real shop products for the links.
App	Smart Buy screen + the verdict card inside the assistant chat.

5. Main endpoints

Endpoint	Does
`POST /m06/verdict`	The verdict (photo or `custom:type:colour`, price, body, answer).
`POST /m06/analyze`	Reads a photo (FashionCLIP): type, colour, confidence.
`POST /m06/shopping-list`	Gaps to buy.
`POST /m06/feedback` · `GET /m06/history/{user}`	Bought / liked → learning; past decisions.
`/m06/wishlist` · `/m06/alternatives` · `/m06/checkin`	Wishlist + price drop, cheaper pieces, 30-day check-in.
`POST /m13/links` · `POST /m18/advise` · `POST /m18/parse-fit`	Buy links + promo; size advice; fit from free text.

6. Files: where they are and what they do

Pipeline files are in `backend/modules/m06_smartbuy/pipeline/`, clients in `backend/modules/m06_smartbuy/clients/`.

Part	File (position)	Lines	Objective
Agent	`backend/modules/m06_smartbuy/agent.py`	256	The Smart Buy agent: photo of a candidate piece → BUY / THINK / SKIP. Collects the facts, decides, asks ONE question when unsure, builds next actions, takes feedback.
Agent	`backend/modules/m06_smartbuy/contract.py`	127	Input / output of BO5 (SmartBuyInput, SmartBuyOutput, Body, Answer, ShopLink...). Built on the shared schemas.
Agent	`backend/modules/m06_smartbuy/identity.py`	67	Smart Buy's agent card for the orchestrator: identity, skills (M13, M18), contract, rules.
Agent	`backend/modules/m06_smartbuy/tool.py`	25	Exposes Smart Buy to the orchestrator (BO8) as the tool `smartbuy`.
Agent	`backend/modules/m06_smartbuy/router.py`	180	HTTP endpoints /m06/...: verdict, analyze, shopping-list, feedback, history, wishlist, alternatives, check-in, labels, card.
Agent	`backend/modules/m06_smartbuy/config.yaml`	—	All the settings: weights, thresholds, real vs mock modules, decision mode, explainer.
Pipeline	`pipeline/redundancy.py`	21	Step 1 — do I already own something similar? (M1 similarity: ≥ 0.85 duplicate, ≥ 0.70 similar).
Pipeline	`pipeline/unlock.py`	6	Step 2 — how many new outfits does it unlock? (M2 with the piece locked).
Pipeline	`pipeline/gap.py`	14	Step 3 — does it fill a gap in the wardrobe? (real gaps from shared/wardrobe_gaps.py).
Pipeline	`pipeline/score.py`	47	Step 4 — weighted score 0–100, verdict and the reasons.
Pipeline	`pipeline/decide.py`	79	Decision layer: rules or learned model, plus XAI (what pushed toward BUY / SKIP).
Pipeline	`pipeline/uncertainty.py`	25	"Not sure" zone: close call or blurry photo → ask ONE question.

Pipeline	`pipeline/whatif.py`	50	Counterfactuals: what would change the answer (price, need, number owned).
Pipeline	`pipeline/wear.py`	19	Predicted wears per year and cost per wear, from similar owned pieces.
ML	`backend/modules/m06_smartbuy/vision.py`	118	Photo analysis with FashionCLIP (zero-shot): type + colour + 512-d image embedding.
ML	`backend/modules/m06_smartbuy/learn.py`	83	Learns the decision (logistic regression) from labelled pieces: rules 71 % → learned 86 % (5-fold CV).
ML	`backend/modules/m06_smartbuy/bandit.py`	78	Personal strictness learned from feedback (Thompson-sampling contextual bandit).
ML	`backend/modules/m06_smartbuy/synthetic.py`	243	Synthetic data: wardrobes, labelled candidates, fictional shops, size charts.
Text	`backend/modules/m06_smartbuy/explainer.py`	71	Writes the explanation (local LLM, template when off).
Features	`backend/modules/m06_smartbuy/extras.py`	222	Wishlist + price-drop check, cheaper alternatives, 30-day check-in (did you wear it? regret?), real labels, accuracy.
Features	`backend/modules/m06_smartbuy/shopping.py`	56	BO5's own task: the shopping list from the wardrobe gaps.
Features	`backend/modules/m06_smartbuy/goals.py`	106	The user's personal goals (e.g. buy less, capsule) that move the score.
Memory	`backend/modules/m06_smartbuy/memory.py`	89	Per-user memory: decisions, feedback, wishlist (one JSON file per user).
Clients	`clients/real.py · http.py · mocks.py · base.py`	316 · 35 · 123 · 19	Calls the teammates' modules for real (M1, M2, M7, gaps), with the mock as a safety net.
Skill M13	`backend/modules/m13_partner_links/service.py · router.py`	100 · 43	Buy links + promo codes in Tunisian shops (TND), click tracking.
Skill M18	`backend/modules/m18_size_fit/service.py · text_fit.py · router.py`	80 · 152 · 47	Size advice from body numbers or fit words, with brand size charts; reads "M at Zara, tight on the shoulders".
BO5 tools	`backend/bo/bo5_purchase/ (cli.py, closet.py, evaluate.py, demo/app.py)`	50 · 76 ·	Terminal test, "my closet" from your own photos, accuracy evaluation, local

		42 · 193	browser test UI (port 8005).
App	`mobile/lib/features/bo5_smart_buy/ui/bo5_smart_buy_screen.dart`	939	The Smart Buy screen: photo / piece, price, verdict ring, why, size, local shops, wishlist.
App	`mobile/lib/features/bo5_smart_buy/data/bo5_smart_buy_repository.dart`	381	Calls M6 + M13 + M18 through the API client.
App	`mobile/lib/features/bo5_smart_buy/logic/bo5_smart_buy_provider.dart`	65	Riverpod state of the screen.
Tests	`backend/modules/m06_smartbuy/tests/ (11 files)`	—	Contract, verdict, explainer, extras, agent memory, real clients, real user.

7. Run and test

```
cd backend
python -m pytest modules/m06_smartbuy -q          # BO5 tests
python -m modules.m06_smartbuy.learn              # rules vs learned (cross-validation)
python -m modules.m06_smartbuy.bandit             # personal strictness simulation
uvicorn bo.bo5_purchase.demo.app:app --port 8005  # local test UI
```